HASIL TUGAS 4 LARAVEL FAISA AL FARREL

isi Route api.php

```php
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\GenreController;
use App\Http\Controllers\AuthorController;
use App\Http\Controllers\BookController;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

Route::apiResource('genre',GenreController::class);

Route::apiResource('author',AuthorController::class);

Route::apiResource('book',BookController::class);
```

isi Controller

a. Author

```php
public function index()
{
    $authors = Author::all();

    return response()->json([
        "success" => true,
        "message" => "Get All Resources",
        "data" => $authors
    ], 200);
}
```

```php
public function store(Request $request)
{
    //Validasi
    $request->validate([
        'name' => 'required|string|max:255',
        'photo' => 'required|image|mimes:jpeg,png,jpg,gif,svg|max:2048',
        'bio' => 'required|string',
    ]);

    //Handle upload foto
    $photoPath = null;
    if ($request->hasFile('photo')) {
        //Simpan file ke storage/app/public/photos
        $photoPath = $request->file('photo')->store('photos', 'public');
    }

    //Simpan
    $authors = Author::create([
        'name' => $request->name,
        'photo' => $photoPath,
        'bio' => $request->bio
    ]);

    //Response
    return response()->json([
        "success" => true,
        "message" => "Data Berhasil Disimpan",
        "data" => $authors
    ], 201);
}
```

```php
public function show($id)    Parameter $id has no t
{
    $author = Author::find($id);

    //Validasi jika data tidak ditemukan
    if (!$author) {
        return response()->json([
            'success' => false,
            'message' => 'Data author tidak ditemukan',
        ], 404);
    }

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data author berhasil diambil',
        'data' => $author,
    ], 200);
}
```

```php
public function update(Request $request, $id)    Par
{
    $author = Author::find($id);

    //Validasi jika data tidak ditemukan
    if (!$author) {
        return response()->json([
            'success' => false,
            'message' => 'Data author tidak ditemukan',
        ], 404);
    }

    //Validasi
    $request->validate([
        'name' => 'required|string|max:255',
        'photo' => 'nullable|string|max:255',
        'bio' => 'nullable|string',
    ]);

    //Update
    $author->update([
        'name' => $request->name,
        'photo' => $request->photo,
        'bio' => $request->bio,
    ]);

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data author berhasil diperbarui',
        'data' => $author,
    ], 200);
}
```

```php
public function destroy($id)    Parameter $id has
{
    $author = Author::find($id);

    //Validasi jika data tidak ditemukan
    if (!$author) {
        return response()->json([
            'success' => false,
            'message' => 'Data author tidak ditemukan',
        ], 404);
    }

    //Hapus
    $author->delete();

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data author berhasil dihapus',
    ], 200);
}
```

b. Genre

```php
public function index()
{
    $genres = Genre::all();

    return response()->json([
        "success" => true,
        "message" => "Get All Resources",
        "data" => $genres
    ], 200);
}
```

```php
public function store(Request $request)
{
    //Validasi
    $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    //Simpan
    Genre::create([
        'name' => $request->name,
        'description' => $request->description
    ]);

    //Response
    return response()->json([
        "success" => true,
        "message" => "Data Berhasil Disimpan",
    ], 201);
}
```

```php
public function show($id)    Parameter $id has no
{
    $genre = Genre::find($id);

    //Validasi jika data tidak ditemukan
    if (!$genre) {
        return response()->json([
            'success' => false,
            'message' => 'Data genre tidak ditemukan',
        ], 404);
    }

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data genre berhasil diambil',
        'data' => $genre,
    ], 200);
}
```

```php
public function update(Request $request, $id)    Pa
{
    $genre = Genre::find($id);

    //Validasi jika data tidak ditemukan
    if (!$genre) {
        return response()->json([
            'success' => false,
            'message' => 'Data genre tidak ditemukan',
        ], 404);
    }

    //Validasi
    $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    //Update
    $genre->update([
        'name' => $request->name,
        'description' => $request->description,
    ]);

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data genre berhasil diperbarui',
        'data' => $genre,
    ], 200);
}
```

```php
public function destroy($id)    Parameter $id has
{
    $genre = Genre::find($id);

    //Validasi jika data tidak ditemukan
    if (!$genre) {
        return response()->json([
            'success' => false,
            'message' => 'Data genre tidak ditemukan',
        ], 404);
    }

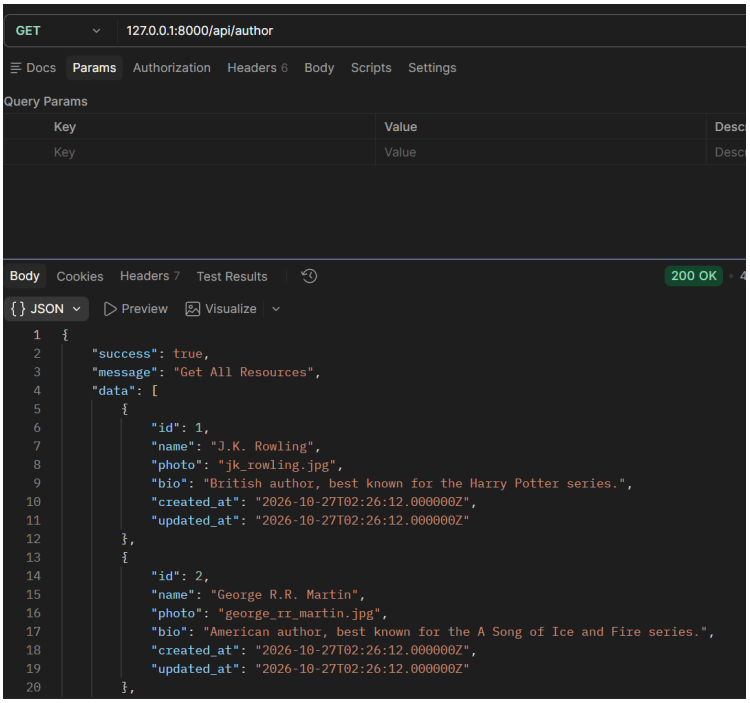
    //Hapus
    $genre->delete();

    //Response
    return response()->json([
        'success' => true,
        'message' => 'Data genre berhasil dihapus',
        'data' => $genre
    ], 200);
}
```

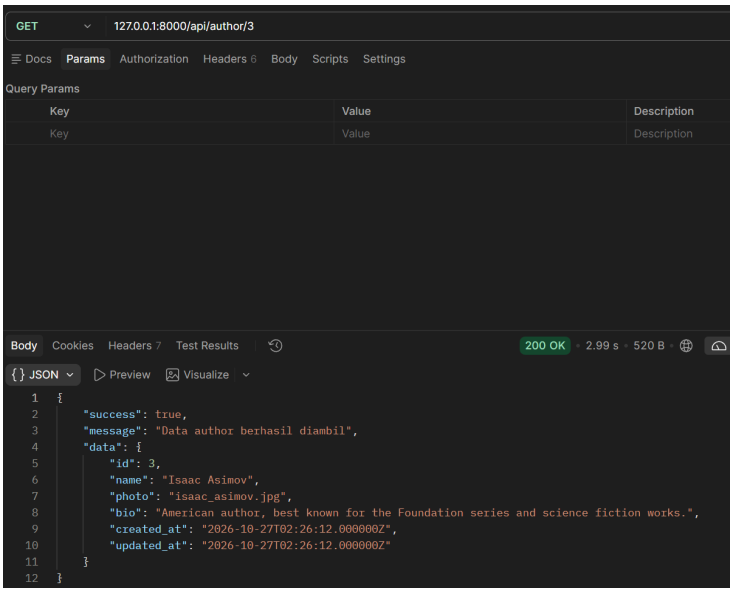
Hasil POSTMAN

a. Author

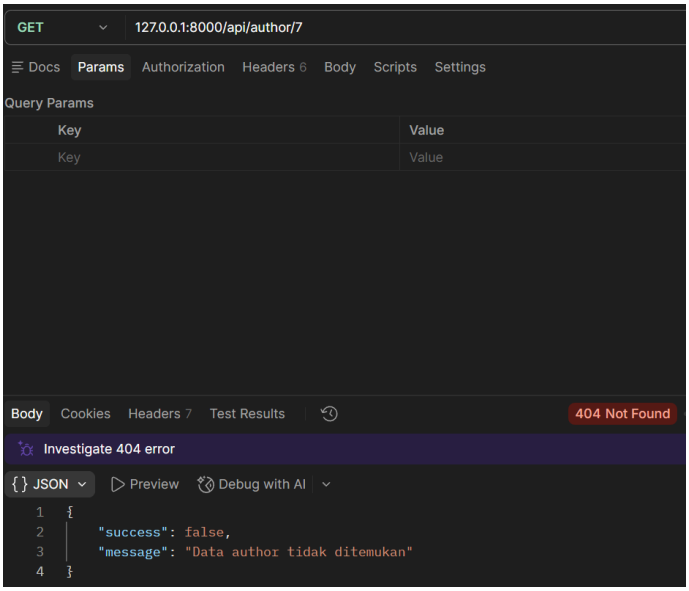
Get all



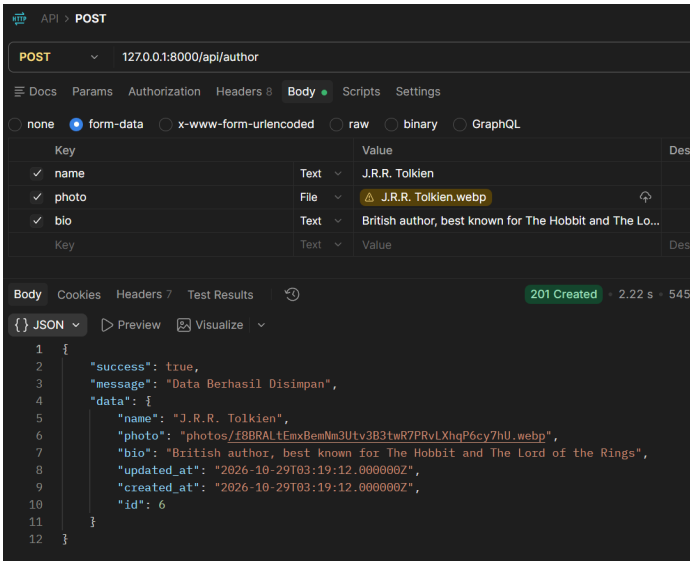
show data id 3



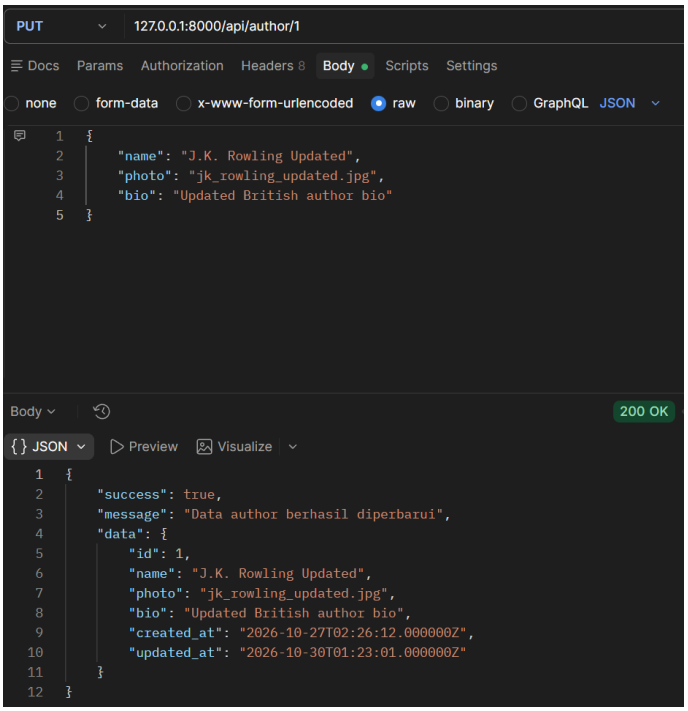
show data tidak ditemukan



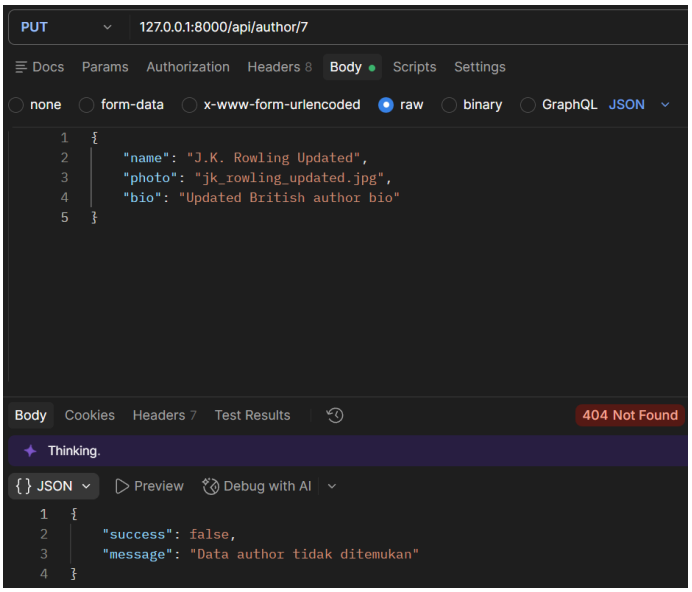
store data baru



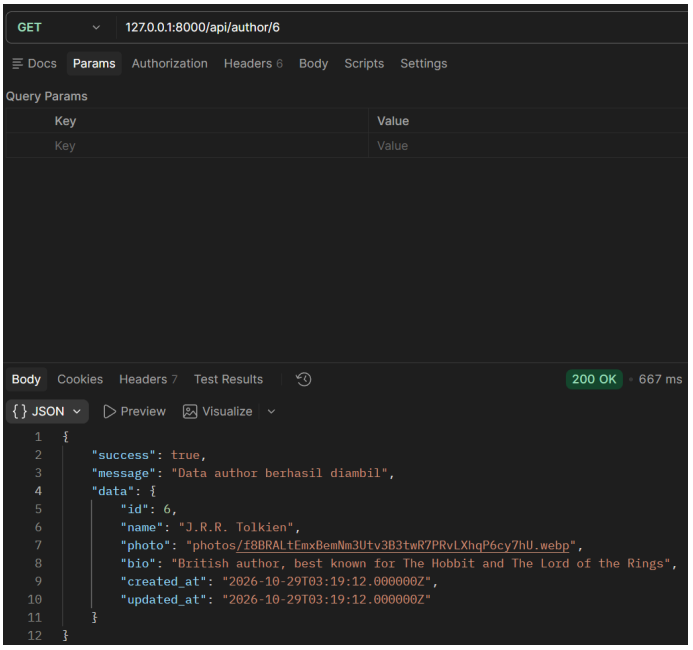
update data id 1

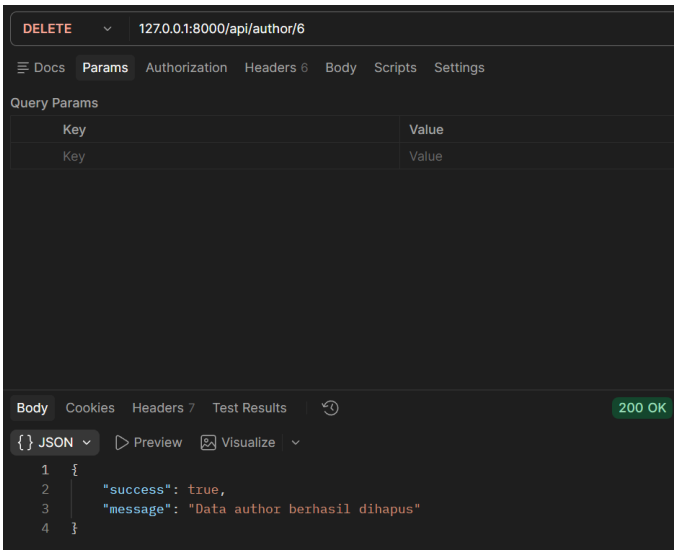


update data tidak diketahui

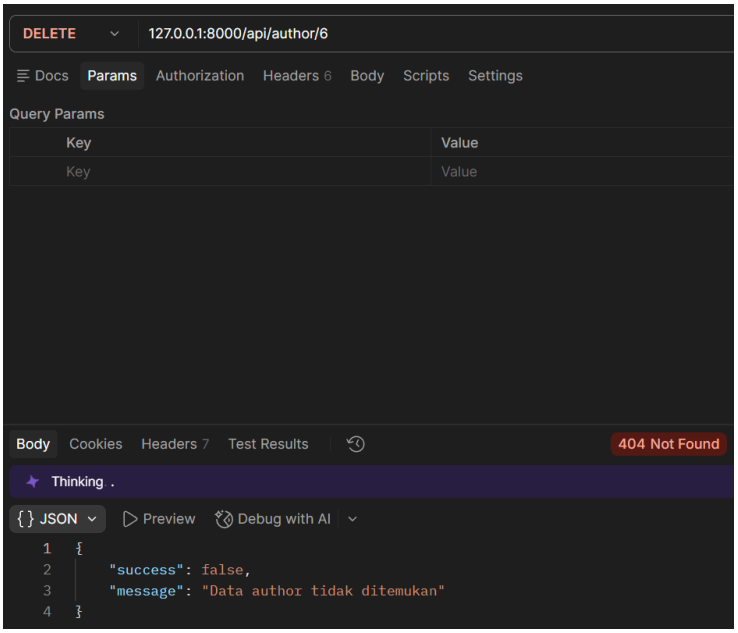


delete data id 6



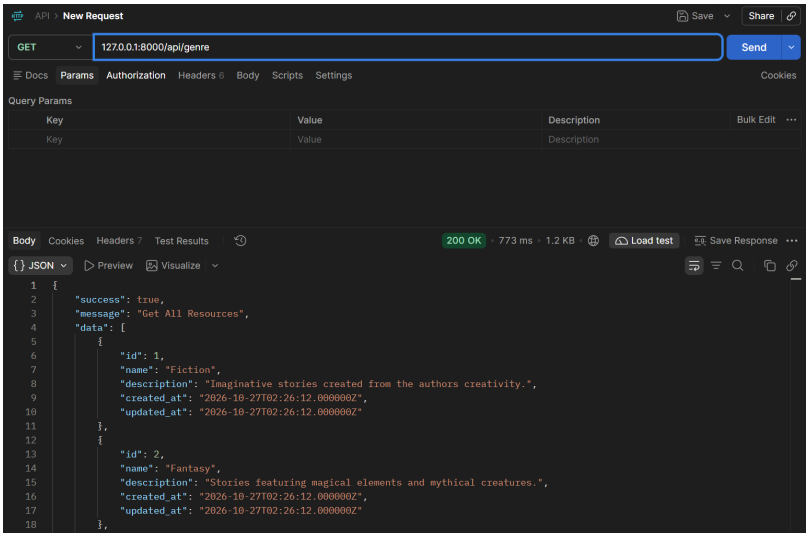


delete data tidak ditemukan

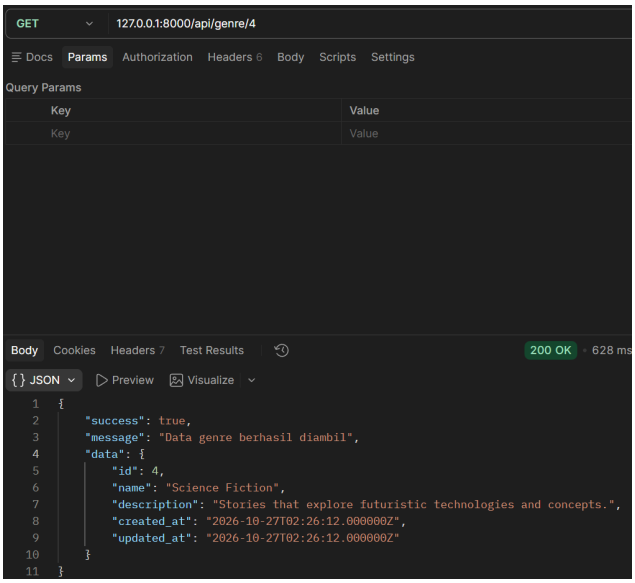


b. Genre

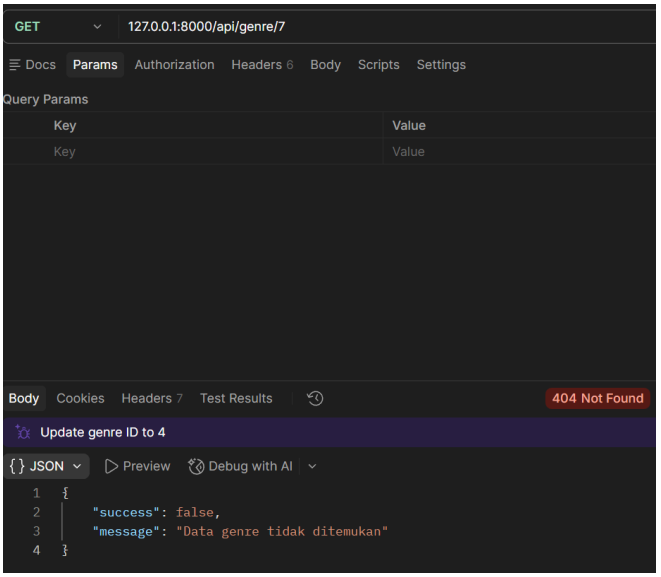
Get all



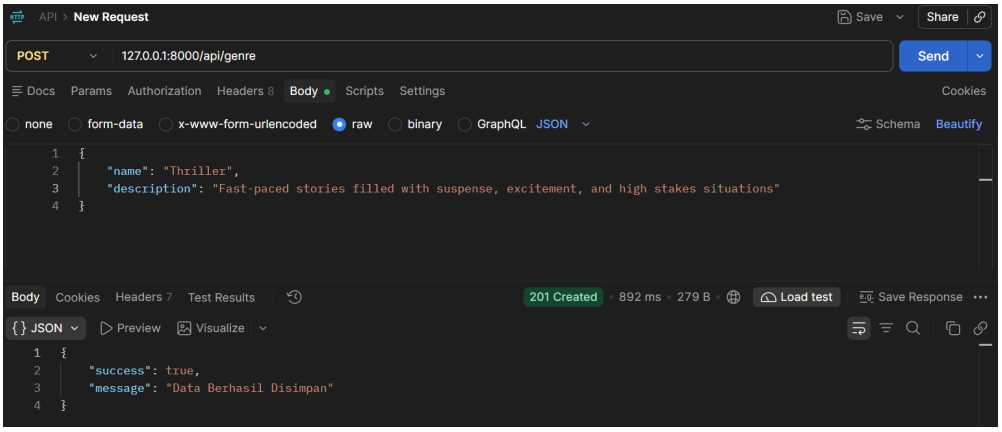
show data id 4



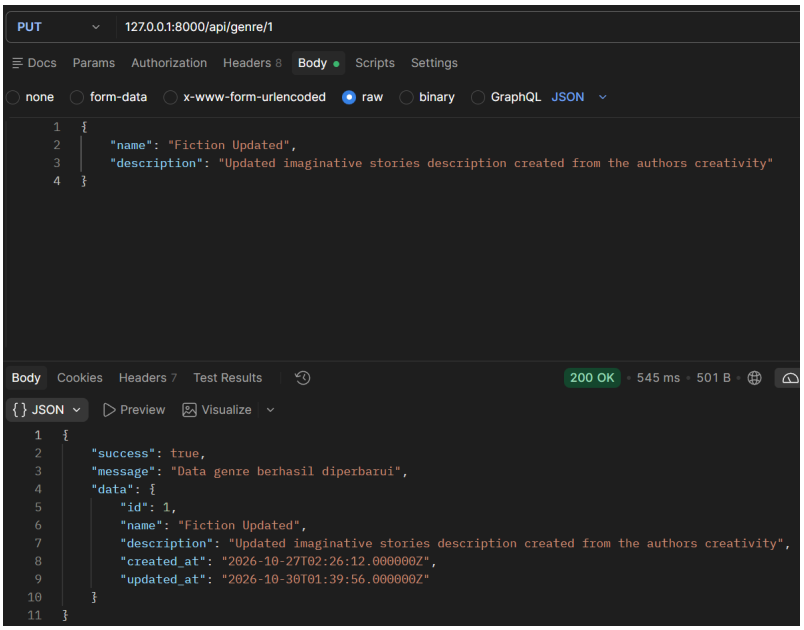
show data tidak ditemukan



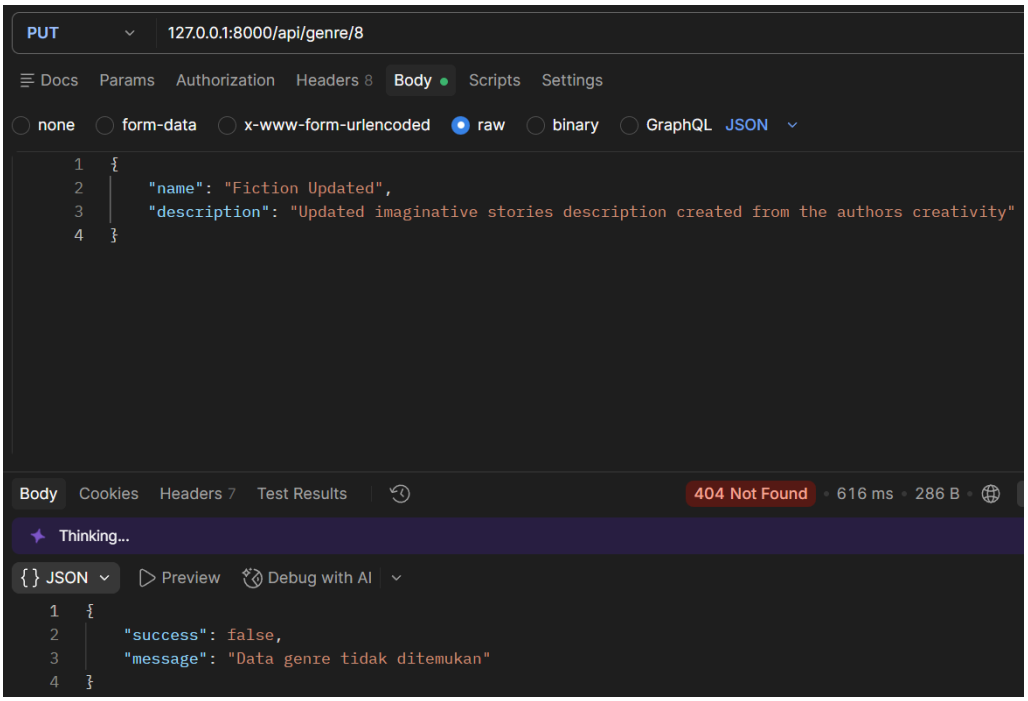
Store data baru



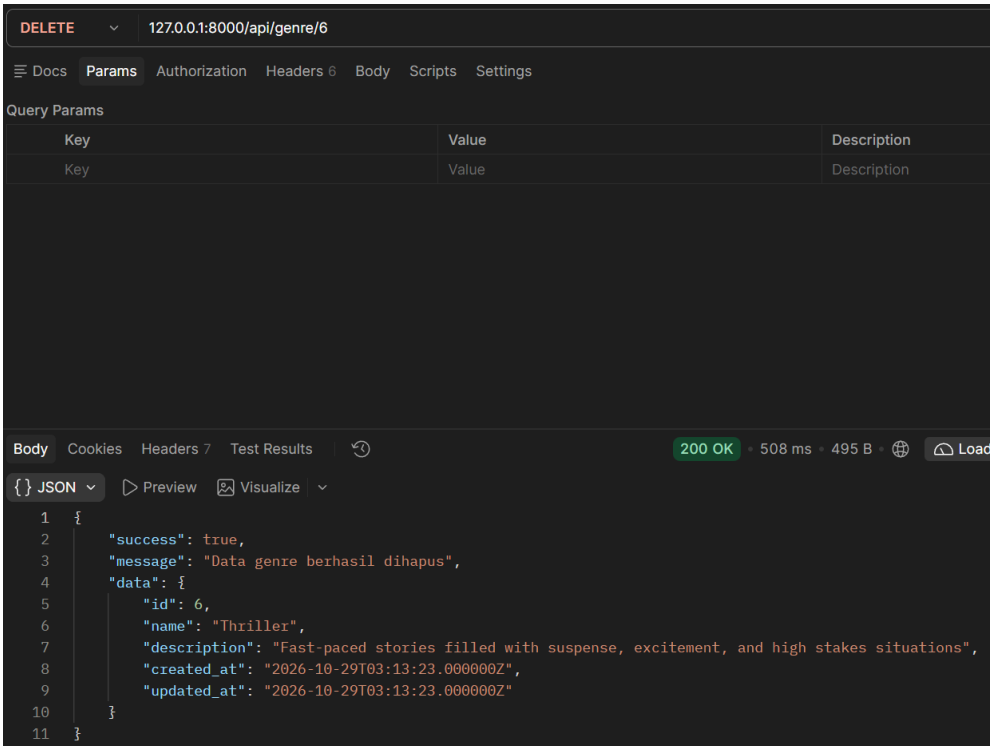
update data id 1



update data tidak ditemukan



delete data id



delete data tidak diketahui

